COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	September 12, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	Silao, Johrel Louie T.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _8.1_ Sorting Algorithms Part 2

1. PROCEDURE OUTPUT

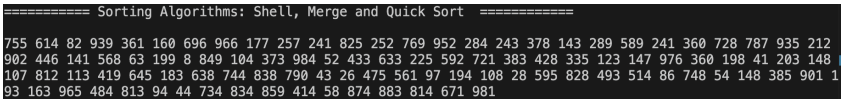
Code + Console Screenshot	  
Observations	This outputs the unsorted array of 100 random elements and serves as the foundation of comparison of elements.

Table 8-1. Array of Values for Sort Algorithm Testing

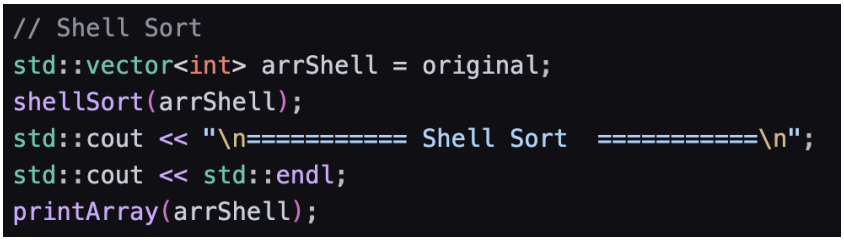
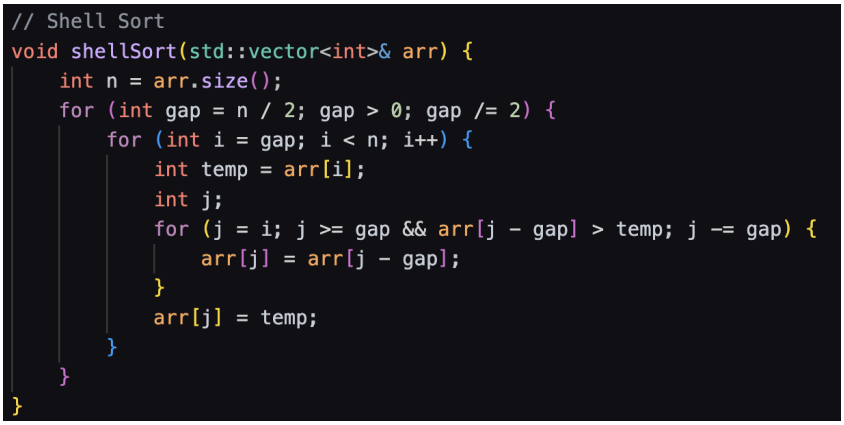
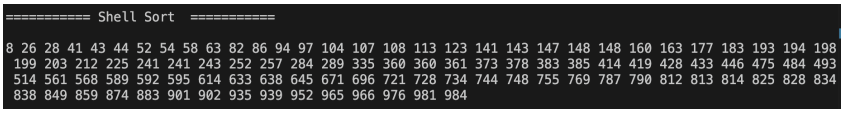
Code + Console Screenshot	  
Observations	This shows the process of Shell Sort in ascending order using the gap based insertion method.

Table 8-2. Shell Sort Technique

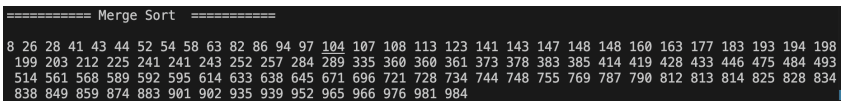
Code + Console Screenshot	  
Observations	This shows the process of Merge Sort that successfully sorts the array.

Table 8-3. Merge Sort Technique

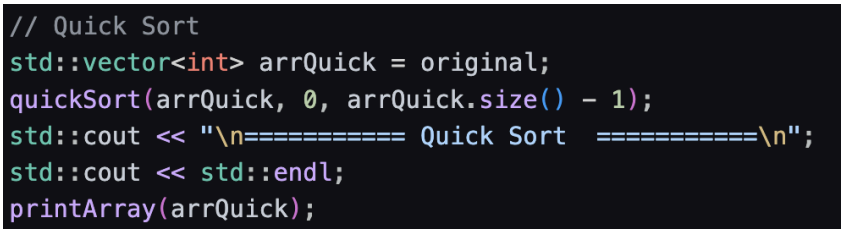
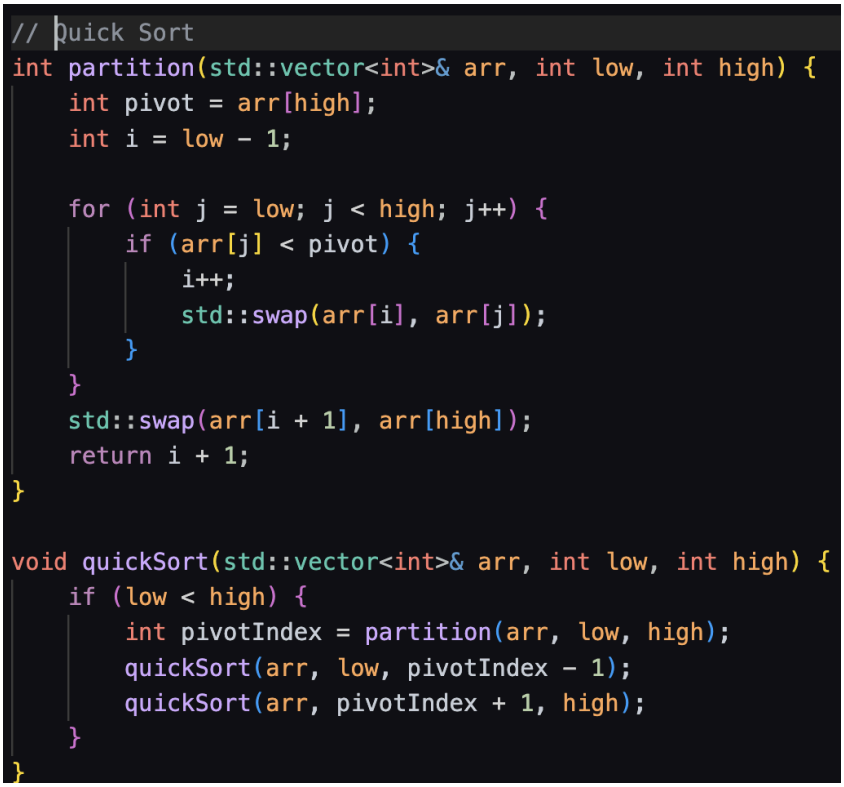
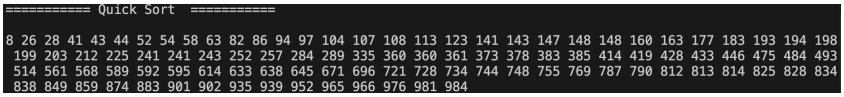
Code + Console Screenshot	  
Observations	This shows the Quick Sort process where it sorts the array using a recursive partition mechanism.

Table 8-4. Quick Sort Technique

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

```cpp
// Supplementary: Problem 1
void supplementaryActivity(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        if (high - low < 10) {
            std::vector<int> temp(arr.begin() + low, arr.begin() + high + 1);
            shellSort(temp);
            for (int i = 0; i < temp.size(); i++) {
                arr[low + i] = temp[i];
            }
            return;
        }

        int pivotIndex = partition(arr, low, high);
        supplementaryActivity(arr, low, pivotIndex - 1);
        supplementaryActivity(arr, pivotIndex + 1, high);
    }
}
```

Figure 1: Code Snippet of main.cpp Problem 1

Observation of Figure 1:

- This implements the modified Quick Sort that manages base cases by assigning small sub lists to Shell Sort to minimize recursion overhead.

```cpp
// Supplementary: Problem 1
void supplementaryActivity(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        if (high - low < 10) {
            std::vector<int> temp(arr.begin() + low, arr.begin() + high + 1);
            shellSort(temp);
            for (int i = 0; i < temp.size(); i++) {
                arr[low + i] = temp[i];
            }
            return;
        }

        int pivotIndex = partition(arr, low, high);
        supplementaryActivity(arr, low, pivotIndex - 1);
        supplementaryActivity(arr, pivotIndex + 1, high);
    }
}
```

Figure 2: Code Snippet of Sorting.h Problem 1

Observation of Figure 2:

- This figure displays the header file that declares the modified Quick Sort.

```
========== Supplementary Activity: Problem 1 ==========
Original Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

Sorted Array:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

Figure 3: Output Snippet of Sorting.h Problem 1

Observation of Figure 3:

- This outputs the problem 1 of supplementary activity where it successfully sorts the random generated integers.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- A sorting algorithm that gives the fastest time performance is Merge Sort. Both merge sort and quick sort splits the input array recursively because it halves the dataset multiple times which will create a recursion tree with a depth proportional $log_2 N$.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.